

# Web V — Travail de session

## Course de frappe multijoueur en temps réel

---

### 1. Mise en contexte

Vous devez concevoir, développer et déployer une application web de **course de frappe au clavier** (dans le style de *TypeRacer* ou *Monkeytype*). Elle servira à faire pratiquer la vitesse de frappe aux étudiants des cohortes plus jeunes.

Les joueurs se retrouvent dans une **salle**, identifiée par un court code (comme Kahoot). Ils s'affrontent en tapant le même texte, et chacun voit la progression des autres en direct sur une piste. Des bots de différents niveaux peuvent compléter la salle, et des bonus de remontée gardent la course serrée jusqu'à la fin.

### 2. Modalités

| Élément       | Détail                                                                     |
|---------------|----------------------------------------------------------------------------|
| Travail       | Individuel                                                                 |
| Checkpoint 1  | Semaine 7   15 % de la note finale (voir section 7)                        |
| Remise finale | Semaine 11   30 % de la note finale (voir sections 8 et 9)                 |
| Outils d'IA   | <b>Permis sans restriction</b> (Claude Code, Codex, Cursor, Copilot, etc.) |

#### 2.1 Utilisation de l'IA

Les entreprises de développement utilisent aujourd'hui massivement les agents IA. Ce travail vous prépare à cette réalité. **Vous restez toutefois entièrement responsable de chaque ligne de code remise.** « C'est l'IA qui l'a écrit » n'est jamais une justification acceptable pour un bogue, une faille ou un choix d'architecture douteux.

**Deux exceptions** : le **nom** et le **logo** du site doivent venir de vous (voir section 4).

#### 2.2 Ambiguïtés

Si une exigence est ambiguë ou incomplète, faites un choix raisonnable et **documentez-le** dans `docs/EXIGENCES.md`. Un choix réfléchi et documenté ne sera pas pénalisé.

#### 2.3 Identifiants des exigences

Chaque exigence porte un identifiant (ex. `COURSE-04`). Vous devrez y faire référence dans votre matrice de traçabilité (section 8).

### 3. Contraintes techniques

- **TECH-01** — Next.js (App Router, version stable actuelle) avec React.
- **TECH-02** — **TypeScript à 100 %**.
  - Aucun fichier `.js/.jsx` dans le dépôt.
  - `strict: true` dans `tsconfig.json`.
  - Aucun `any` explicite (règle ESLint `@typescript-eslint/no-explicit-any` en erreur).
  - Aucun `@ts-ignore` ou `@ts-expect-error` sans commentaire justificatif.
- **TECH-03** — Tailwind CSS pour le style.
- **TECH-04** — Base de données **PostgreSQL**, avec l'ORM de votre choix (Prisma, Drizzle, etc.).
  - Les migrations sont versionnées dans le dépôt.
  - Un script de *seed* crée des données de démonstration : corpus de textes, utilisateurs et historique.
- **TECH-05** — **Déploiement sur un serveur (VPS)**.
  - Vous pouvez louer un petit serveur à vos frais (DigitalOcean, AWS EC2, FlyIO, etc.) ou utiliser un serveur fourni par le cégep.
  - L'application est accessible publiquement en HTTPS.
  - L'application doit être fonctionnelle en production au checkpoint 1 et à la remise finale.
- **TECH-06** — **Visualisation de la progression des participants en temps réel** : la technologie est libre (WebSocket, MQTT ou autre).
- **TECH-07** — Toutes les entrées côté serveur sont validées par un schéma (Zod ou équivalent) : actions serveur, routes API et messages temps réel.
- **TECH-08** — **Coûts**.
  - Les frais du serveur (TECH-05) sont à votre charge, sauf si vous utilisez un serveur du cégep.
  - Tout autre service externe (stockage d'images, service temps réel, etc.) doit utiliser un **forfait gratuit**.
  - Je ne dois rien payer pour corriger. Vous pouvez me fournir une clé d'accès si nécessaire.
- **TECH-09** — Intégration continue avec GitHub Actions. Chaque *push* exécute le lint, la vérification de types (`tsc --noEmit`) et les tests unitaires.
- **TECH-10** — Un fichier `.env.example` documente toutes les variables d'environnement. Aucun secret n'est commité.

### 4. Identité visuelle et design

- **DES-01** — **Nom du site trouvé par vous, sans IA**.
  - Le nom doit être original : vérifiez qu'il n'est pas déjà utilisé par un produit connu.
  - Documentez votre démarche dans `docs/DEMARCHE-CREATIVE.md` : liste des noms envisagés, raison du choix final.
- **DES-02** — **Logo conçu par vous, sans IA**.
  - Joignez vos croquis (photo papier ou tablette) à la démarche créative.

- La vectorisation ou la mise au propre de *votre* croquis dans un outil (Figma, Illustrator, Inkscape) est permise.
- Le logo est utilisé dans l'application et comme favicon.
- **DES-03 — Direction artistique documentée avant le développement.** La démarche créative contient :
  - un moodboard de 3 à 5 références ;
  - la palette de couleurs ;
  - les typographies.
- **DES-04 — Design unique.** Un site qui a l'air « généré par défaut » sera pénalisé. Exemples de signes à éviter :
  - composants shadcn/ui non personnalisés ;
  - dégradé violet générique dans la section d'accueil (*hero section*) ;
  - émojis en guise d'icônes ;
  - mise en page de *landing page* type SaaS sans lien avec le thème.

La **piste de progression** doit être un élément signature du design.
- **DES-05 — Thèmes clair et sombre.**
  - Un sélecteur permet de changer de thème, et la préférence système est respectée par défaut.
  - Aucun flash du mauvais thème au chargement.
  - *Exception* : si votre direction artistique est volontairement mono-thème et qu'un second thème l'affaiblirait, justifiez-le dans la démarche créative.
- **DES-06 — Responsive** : toutes les pages sont utilisables en format mobile (360 px de largeur et plus). Sur mobile, la course est remplacée par un message recommandant un clavier physique.

## 5. Fonctionnalités

### 5.1 Vocabulaire

| Terme              | Définition                                                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Salle</b>       | Espace de jeu identifié par un code. Elle a un créateur (l'hôte), des participants et une configuration. Elle peut enchaîner plusieurs courses. |
| <b>Course</b>      | Une manche jouée dans une salle, sur un texte donné, du décompte jusqu'aux résultats.                                                           |
| <b>Participant</b> | Utilisateur connecté, invité ou bot qui prend part à la course.                                                                                 |
| <b>Spectateur</b>  | Personne présente dans la salle qui suit la course sans taper. Un spectateur ne compte pas dans le nombre de participants.                      |
| <b>Invité</b>      | Personne qui participe sans compte.                                                                                                             |
| <b>MPM</b>         | Mots par minute (WPM). Voir la définition à l'annexe A.                                                                                         |

## 5.2 Comptes et profil

- **AUTH-01 — Authentification.**
  - Connexion par OAuth avec **Discord** et **GitHub**. Les deux fournisseurs doivent être offerts.
  - Une connexion par **nom d'utilisateur et mot de passe** est aussi permise, y compris en production. Elle est notamment utile pour les tests de bout en bout (TEST-03).
  - Une librairie est permise (Auth.js, Better Auth, etc.).
- **AUTH-02** — Un invité choisit un pseudonyme (3 à 20 caractères) avant de rejoindre une salle. Sa session est conservée par un cookie signé.
- **AUTH-03** — Un invité ne peut **pas** créer de salle. Il n'a ni historique persistant ni photo de profil ; un avatar généré est affiché à la place.
- **AUTH-04** — Un utilisateur connecté peut téléverser une **photo de profil**.
  - Types acceptés : JPEG, PNG et WebP.
  - Taille maximale de 2 Mo, validée côté serveur.
  - Image redimensionnée avant l'affichage.
- **AUTH-05** — Un utilisateur connecté peut modifier son pseudonyme d'affichage.
- **AUTH-06** — Page de profil avec les statistiques personnelles :
  - meilleur MPM, MPM moyen et précision moyenne ;
  - nombre de courses et nombre de victoires ;
  - graphique de progression du MPM dans le temps.

## 5.3 Salles et visibilité

- **SALLE-01** — Un utilisateur connecté peut créer une salle et en devient l'**hôte**. L'hôte choisit d'être participant ou seulement spectateur.
- **SALLE-02** — Chaque salle a un **code court unique** de 6 caractères alphanumériques, sans caractères ambigus (0/O, 1/I/L).
- **SALLE-03** — La salle a l'une de ces trois visibilitées :

| Visibilité | Apparaît dans l'explorateur et « Faire une course » | Accès                                |
|------------|-----------------------------------------------------|--------------------------------------|
| Publique   | Oui                                                 | Directement ou avec le code          |
| Sur code   | Non                                                 | Avec le code ou un lien d'invitation |
| Privée     | Non                                                 | Avec un lien d'invitation seulement  |

- **SALLE-04 — Liens d'invitation.**
  - L'hôte génère **un lien distinct par invité**. L'interface permet de générer plusieurs liens, de les copier et de voir leur statut (non utilisé, ou utilisé par qui).
  - Chaque lien contient un jeton aléatoire non devinable (au moins 128 bits d'entropie).

- Un lien est à **usage unique** : à sa première utilisation, il est associé à l'adresse IP de la personne qui l'a utilisé. Une personne ayant une autre adresse IP est refusée.
- La personne associée au lien peut le réutiliser pour **se reconnecter** à la salle.
- Le lien d'un participant expulsé (SALLE-07) est révoqué.
- Tous les liens deviennent invalides à la fermeture de la salle.
- Le code seul ne suffit pas pour entrer dans une salle privée.
- **SALLE-05** — L'hôte configure une **capacité maximale** (minimum 2, maximum 30 participants). Les spectateurs ne comptent pas dans la capacité.
- **SALLE-06** — **Une personne ne peut être que dans une seule salle à la fois.**
  - La règle est garantie côté serveur (contrainte en base de données), pas seulement dans l'interface.
  - Si la personne tente de rejoindre une autre salle, on lui propose de quitter la salle actuelle.
  - Un deuxième onglet ouvert sur la même salle ne crée pas de doublon.
- **SALLE-07** — L'hôte peut **expulser** un participant ou un spectateur. Celui-ci ne peut plus rejoindre cette salle.
- **SALLE-08** — Si l'hôte quitte la salle, le rôle d'hôte passe à la personne humaine *connectée* (participant ou spectateur) présente depuis le plus longtemps. S'il n'en reste aucune, la salle est fermée.
- **SALLE-09** — On ne peut rejoindre une salle que lorsqu'elle est en **attente** ou sur **l'écran des résultats**. On ne peut jamais rejoindre une course en cours.
- **SALLE-10** — Le code de salle est protégé contre la force brute : limite de tentatives par IP, par exemple 10 par minute.

## 5.4 Rejoindre une course

- **JOIN-01** — **Rejoindre par code** : un champ accessible dès la page d'accueil.
- **JOIN-02** — **Explorateur de salles** :
  - liste des salles publiques ;
  - pour chaque salle : nombre de participants et capacité, langue du texte, complexité, hôte et état ;
  - filtres minimum : langue et complexité ;
  - la liste se met à jour sans rechargement manuel.
- **JOIN-03** — **Bouton « Faire une course »** (partie rapide). Le système choisit, parmi les salles publiques joignables et non pleines, **celle qui est la plus proche de sa capacité maximale**. En cas d'égalité, la plus ancienne l'emporte. Si aucune salle n'est disponible :
  - un utilisateur connecté voit un message l'informant qu'aucune salle n'est disponible, et on lui offre de créer une salle publique avec la configuration par défaut, dont il devient l'hôte ;
  - un invité voit un état vide et un message l'informant qu'aucune salle n'est disponible.

## 5.5 Configuration de la course

L'hôte configure les options suivantes dans la salle d'attente. Il peut les modifier entre deux courses.

- **CONF-01** — **Temps maximal** : aucun, ou une durée entre 30 secondes et 10 minutes.

- **CONF-02** — **Langue du texte** : français ou anglais. Ce réglage est **indépendant** de la langue de l'interface.
- **CONF-03** — **Type de texte** :
  - *Cohérent* : passages réels tirés d'un corpus stocké en base de données, par exemple des textes du domaine public.
  - *Aléatoire* : mots tirés d'un dictionnaire.
- **CONF-04** — **Longueur** du texte en nombre de mots.
- **CONF-05** — **Complexité** : facile, moyen ou difficile. Définissez des critères mesurables (longueur des mots, fréquence d'usage, caractères spéciaux...) et documentez-les.
- **CONF-06** — Options à activer ou désactiver : **ponctuation, nombres, majuscules** et **accents** (pour le français).
- **CONF-07** — **Caractères à inclure ou à exclure**. Ce réglage s'applique au texte *aléatoire*. En mode *cohérent*, soit l'option est désactivée, soit seuls les passages compatibles sont choisis. Documentez le comportement retenu.
- **CONF-08** — **Mode d'erreur** :
  - *Correction obligatoire* : le joueur ne peut pas avancer tant que l'erreur n'est pas corrigée.
  - *Libre* : le joueur continue, et l'erreur est comptabilisée.
- **CONF-09** — **Bonus de remontée** activés ou non.
- **CONF-10** — Ajout et retrait de **bots**, avec le niveau de chacun.
- **CONF-11** — Visibilité et capacité (voir SALLE-03 et SALLE-05).
- **CONF-12** — Toutes les personnes présentes dans la salle voient la configuration en temps réel.

## 5.6 Déroulement d'une course

- **COURSE-01** — La course passe par des états explicites, modélisés par une machine à états documentée : EN\_ATTENTE → DÉCOMPTE → EN\_COURSE → RÉSULTATS → (EN\_ATTENTE | FERMÉE).
- **COURSE-02** — L'hôte ne peut lancer la course que si la salle compte **au moins 2 participants, dont au moins 1 humain**. Les bots comptent dans ce minimum ; les spectateurs, non.
- **COURSE-03** — Un **décompte synchronisé** (3, 2, 1) précède la course. Le texte n'est révélé qu'au début du décompte.
- **COURSE-04** — **Zone de frappe** :
  - retour visuel immédiat (caractère correct, incorrect, curseur) ;
  - MPM et précision affichés en direct ;
  - le coller est désactivé.
- **COURSE-05** — **Piste de progression en temps réel**.
  - Chaque participant (humain ou bot) apparaît avec son avatar, son nom, sa position et son MPM courant.
  - Le mouvement est fluide : interpolation encouragée, au moins environ 4 mises à jour perçues par seconde.
  - Le joueur local est clairement mis en évidence.

- Les spectateurs voient la piste, sans zone de frappe.
- **COURSE-06 — Serveur autoritaire.** Le serveur est la source de vérité pour :
  - le temps de départ et de fin ;
  - la progression et le classement ;
  - l'application des bonus.

Il rejette les progressions impossibles, comme un saut de progression ou une vitesse irréaliste.

- **COURSE-07** — Un joueur peut **abandonner** la course, avec une confirmation.
- **COURSE-08 — Déconnexion** : un joueur qui recharge la page ou perd sa connexion reprend là où il était s'il revient dans les 30 secondes. Passé ce délai, il est considéré comme ayant abandonné.
- **COURSE-09** — La course se termine lorsque **tous les participants ont terminé ou abandonné**, ou lorsque le **temps maximal** est écoulé.
- **COURSE-10 — Classement final** :
  1. d'abord, les joueurs qui ont terminé, par temps d'arrivée ;
  2. ensuite, ceux qui n'ont pas terminé avant la fin du temps, par progression ;
  3. enfin, ceux qui ont abandonné, par progression au moment de l'abandon.
- **COURSE-11** — Sur l'écran des résultats, l'hôte peut :
  - **relancer une course** avec les mêmes participants, en modifiant la configuration au besoin ;
  - **fermer la salle.**

Les autres personnes peuvent rester ou quitter.

## 5.7 Bots

- **BOT-01** — Il existe cinq niveaux de bots. Les plages ci-dessous sont indicatives ; vous pouvez les ajuster, mais documentez vos valeurs.

| Niveau        | MPM visé | Taux d'erreur indicatif |
|---------------|----------|-------------------------|
| Noob          | 10–20    | ~12 %                   |
| Débutant      | 20–35    | ~8 %                    |
| Intermédiaire | 35–60    | ~5 %                    |
| Expert        | 70–100   | ~2 %                    |
| Impossible    | 140+     | ~0,5 %                  |

- **BOT-02** — La vitesse d'un bot **varie** pendant la course : accélérations, hésitations, ralentissements sur les mots difficiles. Une vitesse constante n'est pas acceptée.
- **BOT-03** — Les bots **font des erreurs** qui les ralentissent (temps de correction). Ils respectent le mode d'erreur configuré.

- **BOT-04** — Les bots sont clairement identifiés comme tels dans l'interface. Ils sont soumis aux bonus et malus comme les humains.
- **BOT-05** — Le moteur des bots est **déterministe à partir d'une graine (*seed*)**, afin d'être testable unitairement.

## 5.8 Bonus de remontée

L'objectif est de donner une chance de revenir aux joueurs en retard.

- **BONUS-01** — Lorsque les bonus sont activés, un joueur **en retard** reçoit un bonus à des points de contrôle. Est en retard le joueur en dernière position ou à plus de 25 % de progression derrière le meneur. Les points de contrôle sont atteints lorsque le meneur passe 25 %, 50 % et 75 % du texte. Un joueur reçoit au plus 3 bonus par course. La règle exacte est à documenter.
- **BONUS-02** — Il existe **au moins 3 types de bonus**, dont au moins un qui aide le joueur en retard et au moins un qui ralentit le meneur. Exemples :
  - **+3 mots** : ajoute 3 mots au texte du meneur seulement ;
  - **-3 mots** : retire 3 mots au texte du joueur en retard seulement ;
  - **Brouillard** : les prochains mots du meneur sont flous pendant quelques secondes.
- **BONUS-03** — L'activation d'un bonus est annoncée visuellement sur la piste et chez le joueur ciblé.
- **BONUS-04** — Le calcul de la progression et du MPM reste cohérent lorsqu'un texte est allongé ou réduit par un bonus.

## 5.9 Résultats et statistiques

- **RES-01** — **Podium** des 3 premiers.
- **RES-02** — **Tableau complet** : rang, participant, MPM, MPM brut, précision, nombre d'erreurs, temps, statut (terminé, temps écoulé ou abandon) et bonus reçus.
- **RES-03** — **Au moins 2 graphiques**, dont :
  - l'évolution du MPM dans le temps pour tous les participants, sur un même graphique ;
  - les touches les plus manquées du joueur, par exemple sous forme de clavier en carte de chaleur.
- **RES-04** — Un indicateur signale un **record personnel** battu.
- **RES-05** — Les résultats de chaque participant humain connecté sont **persistés**. Cela inclut la série temporelle du MPM, nécessaire pour réafficher les graphiques.

## 5.10 Historique

- **HIST-01** — Chaque course terminée apparaît dans l'**historique** de l'utilisateur connecté, avec pagination.
- **HIST-02** — Cliquer sur une course de l'historique réaffiche sa page de résultats complète.

## 5.11 Internationalisation

- **I18N-01** — Tous les textes de l'interface sont disponibles en **français** et en **anglais** : libellés, messages d'erreur, états vides, métadonnées des pages. Aucune chaîne n'est codée en dur.

- **I18N-02** — Un sélecteur de langue est accessible sur toutes les pages. Le choix est conservé (cookie ou profil). Par défaut, la langue du navigateur est utilisée.
- **I18N-03** — Les dates et les nombres sont formatés selon la langue choisie.

## 6. Qualité

### 6.1 Tests

- **TEST-01** — **Tests unitaires** sur la logique.
- **TEST-02** — **Tests de bout en bout** (*end-to-end*) avec Playwright.
- **TEST-03** — Les tests de bout en bout ne fonctionnent pas avec l'authentification OAuth. Utilisez la connexion par nom d'utilisateur et mot de passe (AUTH-01).

### 6.2 Performance

- **PERF-01** — Lighthouse donne **au moins 90** dans chaque catégorie sur la page d'accueil.
- **PERF-02** — Les mises à jour de progression envoyées au réseau sont **regroupées ou limitées** (par exemple 10 messages par seconde au maximum par joueur). On n'écrit pas en base de données à chaque frappe.
- **PERF-03** — La piste reste fluide avec la capacité maximale de participants.

### 6.3 Accessibilité

- **A11Y-01** — Les textes ont un **bon contraste** avec leur arrière-plan, dans les deux thèmes (niveau WCAG AA).
- **A11Y-02** — Les **bonnes balises HTML** sont utilisées :
  - `<button>` pour les actions et `<a>` pour la navigation ;
  - titres `<h1>` à `<h6>` dans un ordre logique ;
  - `<header>`, `<nav>`, `<main>` et `<footer>` pour structurer les pages ;
  - `<table>` pour les données tabulaires, comme les résultats.
- **A11Y-03** — Les images ont un texte alternatif et les champs de formulaire ont un libellé.
- **A11Y-04** — Le site peut être parcouru au clavier, avec un indicateur de focus visible.

### 6.4 Sécurité

- **SEC-01** — Toutes les actions réservées à l'hôte (lancer, configurer, générer des liens, expulser, fermer) sont **autorisées côté serveur**.
- **SEC-02** — Les téléversements sont validés côté serveur : type réel du fichier et taille.
- **SEC-03** — Les mots de passe sont **hachés** avec un algorithme adapté (Argon2, bcrypt ou scrypt) et ne sont jamais stockés ni journalisés en clair.

## 7. Checkpoint 1 (15 %)

Le checkpoint 1 valide vos **fondations** avant que vous développiez le cœur du jeu : identité visuelle, architecture et déploiement. Vous recevrez une rétroaction que vous pourrez intégrer pour la remise finale.

### 7.1 Livrables du checkpoint 1

1. **docs/DEMARCHE-CREATIVE.md** complet (DES-01 à DES-03) : noms envisagés, croquis et logo, moodboard, palette et typographies. **Le nom et le logo remis au checkpoint 1 sont définitifs.**
2. **docs/ARCHITECTURE.md**, version initiale :
  - diagramme du modèle de données ;
  - machine à états d'une course (COURSE-01) ;
  - ADR sur le choix de la technologie temps réel ;
  - approche prévue pour la gestion des bots.
3. **Application déployée** sur votre serveur (TECH-05), accessible en HTTPS, avec :
  - l'authentification fonctionnelle (AUTH-01) ;
  - la base de données et les migrations (TECH-04) ;
  - la création d'une salle et la possibilité de la rejoindre par code, avec la liste des personnes présentes mise à jour en temps réel (SALLE-01, SALLE-02, JOIN-01) ;
  - le sélecteur de langue et le sélecteur de thème fonctionnels sur les pages existantes (I18N-02, DES-05) ;
  - l'identité visuelle appliquée : logo, palette et typographies.
4. **Intégration continue** fonctionnelle (TECH-09), avec au moins quelques tests unitaires.
5. **docs/EXIGENCES.md**, version initiale : toutes les exigences sont listées avec leur statut actuel.

### 7.2 Grille d'évaluation du checkpoint 1

| Critère                                                                                 | Points     |
|-----------------------------------------------------------------------------------------|------------|
| Cahier des charges                                                                      | 20         |
| Démarche créative et direction artistique (nom, logo, moodboard, palette, typographies) | 20         |
| Architecture : modèle de données, machine à états, ADR temps réel                       | 20         |
| Déploiement fonctionnel en production : HTTPS, authentification, base de données        | 20         |
| Salle créée et rejointe par code, mise à jour en temps réel                             | 10         |
| Intégration continue, langue et thème, qualité initiale du code, matrice des exigences  | 10         |
| <b>Total</b>                                                                            | <b>100</b> |

## 8. Remise finale — livrables

1. **Dépôt GitHub** partagé avec l'enseignant. L'historique de commits doit être régulier et significatif : un dépôt livré en quelques commits massifs sera pénalisé.
2. **URL de production** fonctionnelle.
3. **README.md** :
  - présentation et captures d'écran ;
  - technologies utilisées ;
  - installation locale, avec PostgreSQL via Docker Compose recommandé ;
  - variables d'environnement ;
  - commandes pour lancer les tests ;
  - informations de connexion d'un compte de démonstration (nom d'utilisateur et mot de passe).
4. **docs/ARCHITECTURE.md**, version finale :
  - diagramme du modèle de données ;
  - machine à états d'une course (COURSE-01) ;
  - flux des messages temps réel ;
  - **ADR** (*Architecture Decision Record*) sur le choix de la technologie temps réel et sur la gestion des bots.
5. **docs/EXIGENCES.md** : **matrice de traçabilité** des exigences.

| ID | Statut (complet / partiel / non fait) | Fichiers principaux | Tests associés | Notes et choix |
|----|---------------------------------------|---------------------|----------------|----------------|
|    |                                       |                     |                |                |

*Toute exigence déclarée « complète » alors qu'elle ne l'est pas sera pénalisée plus sévèrement qu'une exigence honnêtement déclarée « partielle » ou « non faite ».*

6. **docs/DEMARCHE-CREATIVE.md** : nom, logo et direction artistique (DES-01 à DES-03), avec les images.
7. **docs/IA.md** (1 à 2 pages) :
  - agents et outils utilisés ;
  - fichiers d'instructions d'agent (CLAUDE.md, AGENTS.md, etc.), qui doivent être commités dans le dépôt ;
  - **3 situations** où l'IA s'est trompée ou a produit du code inadéquat, comment vous l'avez détecté et comment vous l'avez corrigé ;
  - courte réflexion sur votre façon de travailler avec les agents.

## 9. Grille d'évaluation de la remise finale (30 %)

| Critère                                                                                                                      | Points     |
|------------------------------------------------------------------------------------------------------------------------------|------------|
| Cahier des charges (même note que celle du checkpoint #1)                                                                    | 10         |
| Fonctionnalités principales : authentification, salles et visibilité, rejoindre, course en temps réel, résultats, historique | 25         |
| Configuration, bots et bonus de remontée                                                                                     | 10         |
| Design, identité visuelle (nom, logo, direction artistique), thèmes, responsive                                              | 10         |
| Qualité du code : TypeScript, React, architecture, sécurité                                                                  | 15         |
| Tests unitaires et de bout en bout, intégration continue                                                                     | 10         |
| Performance et accessibilité                                                                                                 | 10         |
| Documentation et livrables (README, architecture, matrice, démarche, IA)                                                     | 10         |
| <b>Total</b>                                                                                                                 | <b>100</b> |

### Pénalités possibles :

- application non déployée ou non fonctionnelle en production ;
- présence de JavaScript ou de any ;
- fausse déclaration dans la matrice de traçabilité.

## Annexe A — Définitions de calcul

Pour que les résultats soient comparables d'un projet à l'autre, utilisez ces définitions :

- **MPM (net)** = (nombre de caractères **correctement** tapés ÷ 5) ÷ minutes écoulées
- **MPM brut** = (nombre **total** de caractères tapés ÷ 5) ÷ minutes écoulées
- **Précision** = frappes correctes ÷ frappes totales × 100
- **Progression** = caractères validés ÷ longueur totale du texte (en tenant compte des mots ajoutés ou retirés par un bonus)
- Les espaces comptent comme des caractères.